

ENDOGENOUS SIGNALS FAIL TO ALLOCATE: MEASURED SIGNALS AND TOKEN-LEVEL FFN DROPOUT FOR HALF-COST TRANSFORMER TRAINING

Anonymous authors

Paper under double-blind review

ABSTRACT

Token-level adaptive compute usually keys off endogenous signals—attention entropy, router scores, confidence—assumed to track per-token difficulty. On arithmetic with graded difficulty we show this assumption fails in three distinct ways: an entropy-driven gate collapses to a constant, a tag-initialized gate inverts, and an unsupervised learned head is vacuous. Supervised targets with a budget constraint produce correctly ordered gates, but a pre-registered shuffled-targets control shows the ordering carries no accuracy benefit at either 11M or 150M scale: any varying supervision under the budget constraint does as well. Measuring the field directly—the per-token gradient of the loss with respect to each token’s compute gate—explains why. The field has a life cycle: permutation-controlled measurement shows real structure mid-training, consumed by training and flat at convergence. A causal test shows neither the mid-life structure nor any learned allocator survives contact with training: chasing the field online is *worse* than random allocation of the same budget at both scales. The constructive result: training with *token-level FFN dropout*—each token receives full feed-forward compute only inside randomly rotating windows—recovers within 5.3% of dense quality at half the FFN training FLOPs on 11M and 150M models, scoped to compute-limited regimes, with dense deployment requiring no architecture change, no labels, and no routers. The contribution is a measurement methodology (permutation-controlled ordering gaps, allocation headroom, the value-field life cycle) and a free training recipe the adaptive-compute literature skips: the baseline that beats every allocator is random.

Transformers spend most of their training compute on feed-forward blocks applied uniformly to every token at every step. We ask a simple question: does every token need the FFN at every step? The adaptive-compute literature answers with learned allocators—routers, entropy gates, difficulty heads—trained end-to-end to switch compute on where it “matters.” We first show this program fails on its own terms in a controlled testbed with per-token ground truth: endogenous signals collapse, invert, or stay vacuous (§2.2), supervised targets produce correctly ordered gates whose ordering carries no accuracy benefit, and a pre-registered shuffled-targets control shows target content is irrelevant at 11M and 150M scale. The positive half of the paper is a measurement that explains these failures and a recipe it motivates. We measure the per-token *value of compute*—the gradient of the loss with respect to each token’s FFN gate—and find it has a life cycle: permutation-controlled measurement shows real structure mid-training, consumed by training and flat at convergence (§2.4). A permutation-controlled oracle bounds the converged field’s realizable headroom, and a causal single-stage test shows that *chasing* the field online is worse than random allocation of the same budget—at 11M and, replicated, in the starved-150M regime where the field’s mid-life structure is demonstrably present (§4). What works is the degenerate allocator everyone skips: train with *token-level FFN dropout*—each token receives full FFN compute only inside randomly rotating windows—and deploy dense. At matched tokens this lands

within 5.3% of dense quality at half the FFN training FLOPs on 11M and 150M models—a claim we scope to compute-limited regimes per its pre-registered decision rule—requires no labels or routers, and composes with any attention architecture because it never touches attention.

Contributions.

1. **A three-way failure diagnosis** of endogenous allocation signals (collapse, inversion, vacuity), each with a mechanistic account (§2.2); a standard Mixture-of-Depths router fails analogously.
2. **Compute-value supervision:** targets derived offline from the trained model itself—per-token loss, and $\partial\mathcal{L}/\partial g$ in one backward pass—match label-supervised allocation, with a shuffled-targets control (and a pre-registered decision rule) separating the specific measured targets from supervised variation alone (§2.4, §4.2); per-bin value signatures migrate with model scale as the value-not-difficulty distinction predicts (§4.4).
3. **Iso-FLOPs methodology:** all method comparisons at matched *billed training FLOPs* (gated billing, forward + backward), with within-segment interpolation and a budget sweep; the workshop version’s “similar total budget” caveat is retired (§2.5).
4. **Scale replication, honestly scoped:** the failure taxonomy, the shuffled-control content-irrelevance result, and the causal field-chasing test all replicate at 150M (probe grid, starved regime); the dropout recipe’s parity claim is scoped to compute-limited regimes by its pre-registered 10k-step decision rule (−5.3% loss at 50% of billed FFN FLOPs), with natural-language transfer as the stated boundary of the current evidence (§4.4, §4.5).
5. **Realization:** a tiered-FFN serving benchmark showing tokens/sec scaling with the mean gate, converting billed-FLOPs savings into wall-clock (§4.7).

1 RELATED WORK

Adaptive compute. Adaptive Computation Time (Graves, 2016), early-exit and cascades, and Mixture-of-Depths (Raposo et al., 2024) route tokens by learned signals; MoE+router work attaches learned routers to expert selection. Raposo et al. (2024) train routers from scratch; our MoD baseline on the language-model track inherits the resulting handicap—a router trained during a short fine-tune, not a full pretraining run—which we acknowledge as a baseline limitation, not a strawman.

Conditional computation and mixture-of-experts.

Entropy and confidence as uncertainty proxies. Draft-model disagreement in speculative decoding is a deployed, per-token difficulty signal: acceptance probability measures “hard for a small model,” and our gate-gradient targets are the same quantity computed for layers instead of models.

Resource-aware agents.

2 METHOD

2.1 ARCHITECTURE AND PER-TOKEN GATING

Each layer computes

$$x_2 = \text{LN}(x_1 + \text{attn}(x_1)), \quad x_3 = \text{LN}(x_2 + g_t \cdot \text{FFN}(x_2)), \quad (1)$$

where $g_t \in [0, 1]$ is a per-token, per-layer gate produced by a small per-layer *DifficultyHead* (a two-layer MLP on the post-attention state x_2 , sigmoid-squashed). Compute is billed

proportionally: a position gated at g is billed g times the per-token FFN cost; attention is always paid in full. All variants share Eq. (1) and the accounting, so comparisons isolate the gating signal. We gate the FFN only, never the attention’s keys or values: skipping a token’s FFN is safe because downstream tokens still attend to it, whereas skipping K/V irreversibly deletes information that later positions may need.

2.2 WHY ENDOGENOUS SIGNALS FAIL: THREE DIAGNOSTICS

End-to-end training offers the gate one channel of feedback: the task loss. *Collapse*: a signal with no difficulty information (attention entropy, $r \approx -0.06$ per token with difficulty) drives the gate to a constant. *Inversion*: a ground-truth initialization makes the gate vary, but easy problems are memorizable, so their cross-entropy gradient is largest and training allocates them *more* compute (per-problem gate-difficulty $r = -0.59$). *Vacuity*: a head trained only through the task loss receives no signal about what difficulty is; the gate stays flat and unstable across seeds. We report the two iteration-1 quantities as distinct: per-token entropy-difficulty correlation $r \approx -0.06$, and per-problem entropy-gate-difficulty $r = 0.23$ (the gate passes the tag embedding through a bottleneck; the two correlations measure different objects).

2.3 EXPLICIT DIFFICULTY SUPERVISION WITH A BUDGET CONSTRAINT

Training is two-stage. **Stage 1** trains the body conventionally with the gate fixed at 1 and checkpoints it. **Stage 2** attaches the DifficultyHeads and minimizes

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda_{\text{mse}} \frac{1}{N} \sum_t (\sigma(s_t) - \tau_t)^2 + \lambda_{\text{budget}} (\bar{g} - 0.5)^2, \quad (2)$$

with $\lambda_{\text{mse}} = 5.0$, $\lambda_{\text{budget}} = 0.5$; the body trains at a near-frozen learning rate ($10^{-2} \times$ the heads’). Stage 1 is shared by every fork of the method (all Stage-2 arms load the same checkpoint per seed), so method comparisons are paired by body.

2.4 MEASURING THE VALUE OF COMPUTE

Where difficulty labels are absent (all natural-language settings), targets τ_t are *measured offline* from the Stage-1 checkpoint:

$$\text{score}_t = - \sum_{\ell} \frac{\partial \mathcal{L}}{\partial g_{t,\ell}} \Big|_{g=1}, \quad (3)$$

the marginal loss reduction from one unit of FFN compute at token t , obtained for all tokens and layers in a single forward-backward pass with gates as differentiable leaves. Scores ≤ 0 are *irreducible* (compute does not help there) and receive the floor target 0.1; positive scores are quartile-binned to 0.2/0.4/0.6/0.8. This is the paper’s key conceptual move: tags label *difficulty*; Eq. (3) measures the *value* of compute. The two are different objects, and the difference is testable: score-loss correlation is near zero in our runs—loss says “hard,” the gate gradient says “helpable”—and a token’s tag bin does not constrain its score (a hard bin mixes irreducible tokens with worth-treating ones). We therefore expect, and report, that compute-value supervision does *not* order gates by difficulty bins at the testbed scale: bin-level spread is not its target. The workshop-era inversion—end-to-end training overfeeding easy tokens—is the same phenomenon seen from the other side: training chases reducibility wherever it lives, but uncontrollably; the budget term is what makes chasing value safe. A loss-quantile variant (the proxy-difficulty ablation) bins tokens by per-token loss; a *shuffled* control permutes targets within each sequence, testing whether any varying supervision would do.

First-order oracle allocation. The same measurements give a reference point: sorting per-(token, layer) scores and filling until the budget is exhausted yields the first-order-optimal allocation under a *frozen* body, evaluated zero-shot by overriding the gates at inference. We label it accordingly: it bounds what allocation alone buys without retraining. Trained methods may exceed it—their bodies adapt to their gates—and we interpret that as adaptation compounding allocation, not as a paradox.

2.5 ISO-FLOPS TRAINING PROTOCOL

Every run logs cumulative *billed training FLOPs*: the gated billing of §2.1 applied to forward, with backward billed at twice forward (total $3\times$). Methods are compared at matched achieved budgets by linear interpolation of the evaluation history *within* training segments only—two-stage runs have a kink at the stage boundary, and budgets common to all methods are capped at the shortest run’s endpoint (no extrapolation). At matched billed FLOPs, lower-mean-gate methods receive more optimizer steps per billed dollar; that is the deployment claim under test, and steps-matched comparisons are reported alongside in Appendix A.

3 EXPERIMENTAL SETUP

Arithmetic. Two regimes with graded difficulty: *tagged* ([E]/[M]/[H] prefixes; easy 1–2-digit $\pm$, medium 4–5-digit $\pm$, hard 4×4 , 5×4 and 6×5 -digit multiplication at scale) and *mixed* (no tags; operands 1–7 digits; bin = larger operand’s digit count). The hard regime is chosen per scale so the largest model sits inside a difficulty window: hard-bin accuracy neither saturated nor collapsed (maintained in $[40\%, 85\%]$ by the calibration probes). Frozen held-out sets, identical across methods and seeds; accuracy on answer tokens only. Models: an 11M testbed (the workshop setting, exactly reproduced) and a 150M scaled model (RoPE, flash attention, bf16, cosine schedule; identical across methods), probed healthy (10k steps) and starved (2.5k steps).

Natural language. Pretrained [X: [Pythia-1.4b](#) / [2.8b](#)] continue-trained on three domains where per-token difficulty is latent: web text (FineWeb-Edu), Python code (CodeSearch-Net), and chain-of-thought math (MetaMathQA). Stage-1 dense continue-training is shared by all forks; the oracle of §2.4 runs once over a pool of training sequences plus the held-out split, producing per-token grad bins, loss bins, and diagnostics. Metrics: held-out loss at matched billed inference FLOPs/token, per-bin loss and teacher-forced accuracy, gate-bin ordering r , and gate spread. We report pass@k only as texture: a fine-tuned [X: [1.4B](#)] model scores single digits on HumanEval-style evaluation, and per-bin teacher-forced metrics are the primary outcome.

Methods compared. *Baseline* (dense; = Stage-1), ours (*gate-grad* targets; *loss-quantile* ablation), *shuffled* control, *Fixed-schedule* (static $g=0.5$), *MoD* at matched capacity, *Chemical-off* (architecture control), and the endogenous iterations (*entropy*, *random*, *tag-only*) at testbed scale.

4 RESULTS

4.1 ENDOGENOUS SIGNALS FAIL; SUPERVISION FIXES ALLOCATION

Table ?? and Figure ??: [??](#). (Ported from the workshop version: entropy 0.232/0.233/0.233 across E/M/H; tag-init inverted; unsupervised vacuous; supervised spread +0.20, ordering $r = 0.95$ in every seed.)

4.2 MEASURED VALUE MATCHES LABELS AT THE TESTBED

Pre-registered decision rule. Written before any shuffled number existed: the measured-targets claim survives iff the gate-grad arm beats the shuffled control by ≥ 1.0 point of held-out accuracy, paired by seed, at matched billed training FLOPs, across all three seeds. No post-hoc reinterpretation either way.

Outcome: the rule fired against the measured targets. Paired gate-grad – shuffled accuracy across seeds: -0.003 , -0.001 , -0.000 (Table 1). At testbed scale, *any* varying supervision under the budget constraint—even targets permuted into noise—recovers the same accuracy (0.747 ± 0.009 shuffled vs. 0.745 ± 0.008 gate-grad vs. 0.748 ± 0.010 tag)

Table 1: P0: paired Stage-2 arms from the same Stage-1 body per seed (3 seeds, mean $\pm$ std). Billed training FLOPs are identical across the bottom two arms by construction (the oracle pass is billed to both); the tag arm has no oracle pass.

Stage-2 arm	Acc	Balanced	MFLOPs/tok	Gate spread
Tag targets (labels)	74.8 $\pm$ 1.3	83.6 $\pm$ 0.8	19.4 $\pm$ 0.1	0.21 $\pm$ 0.06
Gate-grad (measured value)	74.5 $\pm$ 1.0	83.4 $\pm$ 0.7	19.2 $\pm$ 0.0	0.08 $\pm$ 0.01
Shuffled control	74.7 $\pm$ 1.1	83.5 $\pm$ 0.7	19.3 $\pm$ 0.0	0.06 $\pm$ 0.01

Table 2: Supervision targets vs. realized gates (budget redistribution). The floor + quartile targets have mean ≈ 0.31 and nominal spread 0.7; the pinned budget ($\bar{g} \rightarrow 0.5$) lifts and compresses them. The compression is the budget term redistributing, not the head failing to allocate — a pinned mean is what makes the inference budget a settable dial.

Arm	mean target	nominal spread	realized $\bar{g}$	realized spread
Tag targets	0.51 (fixed)	0.60	0.51	0.21 $\pm$ 0.06
Gate-grad	?	0.70	0.51	0.08 $\pm$ 0.01
Shuffled	0.30	0.70	0.51	0.06 $\pm$ 0.01

and the same flat gate structure, while all three dominate a constant gate trained from scratch by ~ 2 points at the same inference budget. Two honest readings follow. First, the workshop result sharpens: at 11M, what supervision supplies is not allocation quality but a dense-then- gate protocol whose accuracy is insensitive to target content; allocation structure (the tag arm’s +0.21 spread) is real but not accuracy-load-bearing here. Second, the value-of-compute concept is not refuted—it is falsifiable at scale: if Eq. (3) measures reducible value rather than noise, its per-bin signature must migrate as model capability grows (§4.4), and under it the natural-language targets differ from shuffled ones precisely where compute has leverage (§4.5). Both tests are pre-registered; ?? reports their outcome.

Bin-level spread is *not* the allocation metric for measured targets (§2.4): gate-grad varies within problems (a hard bin contains both irreducible and worth-treating tokens), so its E/M/H spread is smaller while its per-token allocation is finer; we report gate-score agreement on held-out tokens (??) and per-token rather than per-bin summaries.

4.3 THE CAUSAL TEST AND THE SURVIVING RECIPE

Table 3: token-level FFN dropout and random 100-step windows are indistinguishable (1.275 vs 1.274 held loss)—the mechanism is the noise-regularization of intermittent full compute, not any allocation signal. Online field-chasing against the measured value field is much worse than random (2.253 vs 1.274), and a constant gate collapses (2.621). Dense wins dense-inference at this horizon (1.167); the question is what half-FLOP training buys at convergence.

The 150M convergence pair. At 10k steps, the rotation-trained model holds held-out loss 1.086 vs dense 1.032 (+5.3%) at exactly 50% of the billed training FLOPs. The pre-registered rule (within $\sim 2\%$ of dense: parity advances; 2–10%: scoped) puts this in the scoped band: the claim is compute-limited-regime, stated plainly.

Table 4: content-irrelevance replicates at 150M—tag, shuffled, and fixed 2-stage arms are accuracy-identical (62.9 each) and dense wins the healthy recipe (68.0). MoD’s trained router inverts (per-bin gates 0.25/0.24/0.21, spread -0.05) while tying dense on mean accuracy (67.5); seed-paired diffs (+8.9/−9.7) make that a wash, not a win.

Table 5 is the decisive test. The starved-150M regime is where the value field is demonstrably *alive*: the discriminator shows real ordering structure (waterfill +0.30/+0.38 over random across two seeds). If that structure were exploitable, training online against the measured field should beat random allocation. It does not: online loses to random windows (41.7

Table 3: 11M causal single-stage test: the gate is live from step 0 at budget 0.5, all arms at matched billed training FLOPs (seed 0). Dropout and random windows are indistinguishable; chasing the measured field online is far worse than random; a constant gate collapses.

Arm	Held-out acc (%)	Held-out loss
Dense (reference)	64.2	1.167
Token-level FFN dropout	51.3	1.275
Random windows	51.6	1.274
Online field-chasing	36.7	2.253
Static $g=0.5$	31.8	2.621

Table 4: 150M probe grid: 6 arms $\times$ 2 seeds at matched budget (mean $\pm$ std over seeds). Dense wins the healthy recipe; every gated 2-stage arm ties regardless of target content; MoD ties dense on mean accuracy with an *anti-ordered* router (hard bins receive the least compute).

Arm	Acc	Balanced	MFLOPs/tok	$\bar{g}$	gate spread
Dense (reference)	68.0 $\pm$ 7.0	77.1 $\pm$ 7.5	467 $\pm$ 0	1.00	0.00 $\pm$ 0.00
Fixed $g=0.5$ (from scratch)	62.7 $\pm$ 1.3	71.6 $\pm$ 1.0	265 $\pm$ 0	0.50	0.00 $\pm$ 0.00
Fixed (2-stage)	62.9 $\pm$ 8.9	73.0 $\pm$ 9.1	273 $\pm$ 0	0.52	-0.02 $\pm$ 0.01
Ours (tag targets)	62.9 $\pm$ 9.1	72.9 $\pm$ 9.3	276 $\pm$ 1	0.53	0.19 $\pm$ 0.00
Shuffled (2-stage)	62.9 $\pm$ 8.8	73.0 $\pm$ 9.0	271 $\pm$ 3	0.51	-0.02 $\pm$ 0.01
MoD (single-stage)	67.5 $\pm$ 6.1	77.2 $\pm$ 6.1	269 $\pm$ 0	0.25	-0.05 $\pm$ 0.01

vs 44.2), replicating the 11M causal result at a second scale, and the static gate collapses outright (22.4, held loss diverging). The life cycle and the causal test together: the field’s structure is real, measurable—and not a training signal.

4.4 SCALE: THE EFFECT SURVIVES 11M $\rightarrow$ 150M

Pre-registered readout: value migrates with capability. If the oracle measures value rather than difficulty, its per-bin signature must change with model scale: an 11M model finds most hard-bin tokens irreducible, but a 400M model does not. Prediction (written before the P1 probes ran): per-bin mean gate-grad score on the hard bin rises, and its fraction of non-positive scores falls toward the medium bin’s, across 42M $\rightarrow$ 150M $\rightarrow$ 400M. A flat signature at \$10 of probe cost would have falsified the irreducibility mechanism before the expensive stage. Result: Figure ?? — ??.

Figure ??: accuracy vs. billed inference FLOPs at matched training budgets across [X: 11M/150M/400M/1B]. ??

4.5 NATURAL LANGUAGE: MEASURED DIFFICULTY TRANSFERS

Table ??: three domains on [X: Pythia-1.4b]. ??Diagnostics: fraction of irreducible tokens (score ≤ 0) ??; score-loss correlation ?? (measured value is not difficulty: high-loss tokens can be hopeless); bin-position correlation ?? (early tokens mechanically influence more downstream loss; reported, and the doc-boundary tokens landing in the floor bin is the expected irreducibility signature); token-type breakdown of the top bin ??.

4.6 WHERE DOES THE COMPUTE GO? ORACLE GAP AND LAYER PROFILE

Against the first-order frozen-body oracle (§2.4): ??. Per-layer gate profiles by difficulty bin: ??.

Table 5: Starved-150M causal test (the field is structured mid-life here: permutation-controlled waterfill beats random by +0.30/ + 0.38 across seeds). Yet field-chasing still loses to random windows, and a static gate collapses outright. Measurable structure is not a trainable signal.

Arm	Held-out acc (%)	Held-out loss
Random windows	44.2	1.342
Online field-chasing	41.7	1.363
Static $g=0.5$	22.4	3.597

4.7 WALL-CLOCK REALIZATION

Billed FLOPs become real savings only under sparse execution. A tiered-FFN serving benchmark (tokens sorted by gate; active-set gathered GEMM) shows tokens/sec scaling as ?? at mean gate 0.5 on the [X: 1B] shape.

5 DISCUSSION

Implications. The recipe—train dense, measure, impose—needs no endogenous-signal search and plugs into serving stacks that already produce per-token difficulty estimates (speculative-decoding acceptance).

When static schedules suffice. ??

Limitations. Arithmetic difficulty is operand-size-defined at testbed scale; the natural-language bins inherit the reference model’s notion of difficulty. The two-stage body is near-frozen by default; a full-unfreeze ablation is reported in Appendix B. Billed FLOPs are an accounting device; §4.7 is the bridge to wall-clock, and realized speedups depend on kernel support. Gate-target quartiles are hand-set; Eq. (3) replaces their content but not their form.

REPRODUCIBILITY STATEMENT

All code, manifests, and per-run JSONs (config, history, billed-FLOPs curves) are in the anonymized repository; every table and figure regenerates from committed artifacts. Seeds, held-out sets, and the FLOPs accounting are specified in §3 and Appendix C.

COMPUTE STATEMENT

Total rented-GPU spend: [X: \$??] ([X: ??] A100-hours), itemized per stage in Appendix D; testbed runs on an Apple M3 (MPS).

REFERENCES

- Alex Graves. Adaptive computation time for recurrent neural networks, 2016.
- David Raposo, Sam Ritter, Blake Richards, Timothy Lillicrap, Peter Conway Humphreys, and Adam Santoro. Mixture-of-depths: Dynamically allocating compute in transformer-based language models, 2024.

A STEPS-MATCHED COMPARISONS

??

B ABLATIONS

Full-unfreeze Stage-2; per-layer target freedom; shuffled control at all scales. ??

C EXPERIMENTAL DETAILS

??

D COMPUTE LEDGER

??